Short Requests, Saved Files and a Later Check

A class reading from the public wavves pages and the repository README

A person starts with a short request. The repository README says the `/wavves` entry reads the request, checks the home, picks a playbook and runs that skill.[1] The published examples are short tasks in ordinary language.

The public listing describes a worked session on that entry.[2] The request charters a lane named docs-sweep for a public docs refresh and runs a first `/mod-check` on the written plan. The moderator wrote `waveset.md` under the lane home. Parallel runners in that review wave returned blockers along with gaps. The build step stayed unstarted during the open review wave.

The class has been talking about a context window that fills up and about recaps that drop detail. The listing answers the window problem this way. A person charters lanes so long work runs as waves with state saved on disk. An orchestration agent runs the waves in the background. Runners do the scoped execution-layer work assigned by that agent. The next moderator term hydrates from files on disk with nothing carried over in the context window.

The class pattern is a short live window plus lasting files, with a later check before the next stretch opens. The later check is `/remeasure` on the JSON and the log in `gate-captures`. On pickup the next term also runs the pickup check and can run the rotation history audit. The public site describes that same folder layout and says the control-surface video uses invented sample files.[3]

Files a Fresh Term Reads

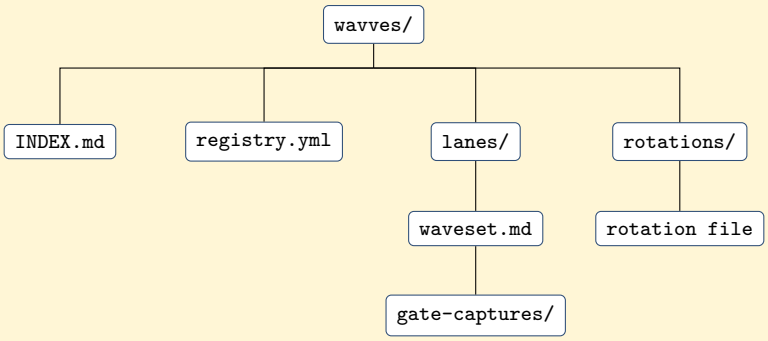


Figure 1. Home files named across the sources. The public site shows `INDEX.md`, lane folders, a waveset, `gate-captures` and a rotations folder. The README adds `registry.yml`, `AGENTS.md`, `step-log.md`, dispatch files, `findings/` and `decisions/`. Lines show which folder holds which file. The site says the demo sample files are invented.

Two Orders the Sources Actually Give

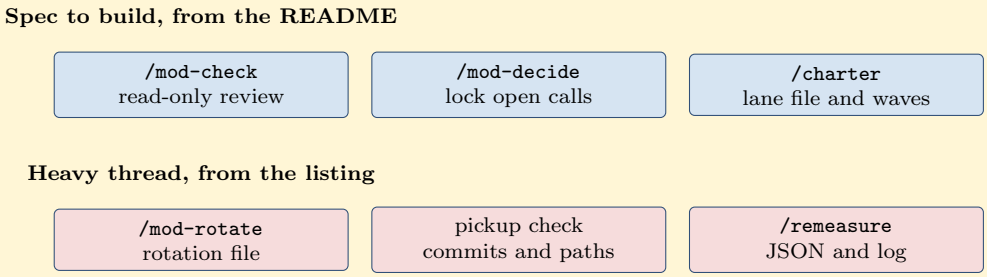


Figure 2. Left to right within each row. The README gives the spec to build order as `/mod-check`, then `/mod-decide`, then `/charter`. The listing gives the heavy-thread order as a rotation file, then the pickup check, then a remeasure of `gate-captures`. These are two sourced orders. The docs-sweep session left the build step unstarted while the review wave was open.

[1] Repository README in this repo, version line 0.5.0.
[2] https://www.agensi.io/skills/wavves
[3] https://wavves.aimez.ai/

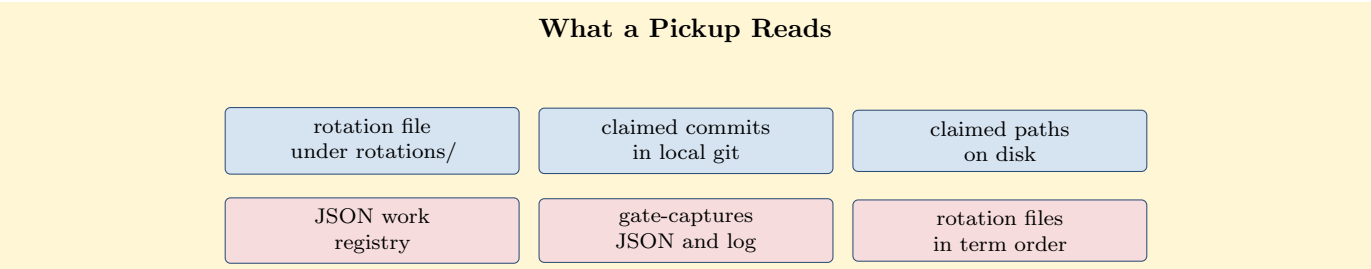


Figure 3. The listing says one pickup script tests claimed commits against local git and looks for claimed paths. A second check covers the JSON work registry. The same pickup remeasures `gate-captures`. The rotation history audit then reads rotation files in order. A fix row for script messages sits in five recovery playbooks.

Published usage

The listing's example session prompt is a short request.

```
/wavves charter a lane named docs-sweep
for the public docs refresh
and run a first /mod-check on the written plan
```

The listing says that session chartered the lane, wrote `waveset.md`, ran a read-only review wave and left the build step unstarted.

The README prints prompt shapes for the same skills. It also says the paths and metrics in `examples/usage.md` are fictional. That file is not in this repository. The lines below are prompt shapes from the README, not measured runs.

```
/wavves set up in this repo, then audit our README for drift.
/mod-check the landed spec.
/mod-decide lock the open calls.
/wavves hydrate from the rotation paste and tell me what's active.
```

Leaf skills from the README. The listing also names `/remeasure`, which the README skill table does not use.

Skill	When the sources say to use it
`/wavves`	Default entry. Matches a playbook and runs it.
`/wavves-init`	Home setup only.
`/charter`	Charter a lane and dispatch waves behind gates.
`/mod-check`	Read-only review of a landed spec or plan.
`/mod-decide`	Lock open calls before a build charter.
`/layover`	Read-only audit of a multi-root workspace.
`/mod-rotate`	Hand a heavy thread to the next moderator term.
`/mod-kick`	Move to another machine with an allowlist and a short paste.
`/shrug`	Carry out recommended actions from a verdict.
`/remeasure`	Listing name for turning a done claim into a rerunnable gate.

Table 1. Skills named in the repository README, plus `/remeasure` from the public listing and the public site. `/set-key` is in the README as a local secret helper and is omitted here so this reading carries no key name and no setup command for a secret.

Glossary

Wavves terms stay as the sources spell them. The sentence after each term is the ordinary class meaning, limited to words this class has already used.

moderator The thread a person talks in. Waves go out from it and grades come back. The README also names this operator-facing thread O0.

moderator term A fresh thread that takes over a heavy one. Term ids look like `r02`. The new term starts without the old transcript.

playbook The named route `/wavves` picks from a short request. The README lists bootstrap, charter-lane, check, decide, layover, paragraph-tunnel, proof-before-accept, rotate, kick, pickup, proceed and shrug.

lane One stream of work with its own folder. One bounded stretch, with its own charter and its own finish checks.

wave One round of parallel work inside a lane. The moderator numbers waves from W1 up in `waveset.md`.

waveset.md The lane file. The README calls this charter an alignment packet. It holds scope, grounded facts, locked decisions, file boundaries and acceptance checks.

alignment packet The written directive for one lane. Another process can read it without a person restating the chat.

standing home The `wavves/` folder. A fresh term hydrates from these files and not from a transcript. `INDEX.md` is the fast pickup page.

registry.yml The map of chartered lanes and their status.

dispatch file The per-wave paste block. The README names `dispatch-wN.md`.

orchestration agent The background runner of a wave. The README also calls this task the wave orchestrator. It fans out charge workers and writes the rollup and the gate.

charge worker One background task for one bounded piece of the wave. The README says it does not commit and does not ask the operator questions. The listing calls the same role a runner on the execution layer.

gate-captures JSON and a log for a gate, on disk beside the lane. The README says a hand-written summary is not a gate capture.

/remeasure Turns a done claim into a gate with a claim and a check someone else can rerun. The grade comes from someone other than the author. The listing says the bundled script rejects self-grading, missing fields and placeholder evidence.

disk gate The README rule that a lane advances after a runnable check writes pass or block evidence under `gate-captures`. Failed gates stay failed. Remediation loops are capped. The sources do not give that cap as a number.

proof-before-accept A named proof job on a product lane. The README says a process-only accept cannot clear when proof is required.

paragraph tunnel A mid-render structural gate for one outbound field. The README says it is not a voice copy.

/mod-check A read-only review wave against a written plan. The README verdict words are GO, REVISE and BLOCK, with recommended actions.

/mod-decide Locks open product calls one at a time and writes decision files. The README says locks then sync to the waveset, the dispatch file and the registry so a later wave does not reopen them. That sync is the authority propagation the README names.

/mod-rotate Writes a rotation file and hands the thread to the next term. Older rotation files never change. The next term can audit the chain.

rotation history audit Reads rotation files in order. Gaps in the numbering and breaks in the chain of terms get an ERROR line. The same run flags deleted rotation files, claimed commits missing from git and claimed paths missing from disk.

pickup check Tests the rotation file against local git and against paths on disk. It also checks the JSON work registry. A fix row for a script message sits in five recovery playbooks.

hydrate The fresh term reads the home and starts from those files. Nothing is carried over in the context window.

/mod-kick A move to another machine. The README says the term publishes an allowlist and a short paste. It is not a substitute for rotate or for pickup.

/layover A read-only audit of sibling repos in a multi-root workspace. It writes one report and does not change those repos.

/shrug Carries out recommended actions. The README says a bare shrug stays on the narrow AUTH-10 proceed path. A closed phrase widens it. The listed phrases are all still standing, queue all standing and move, plus proceed all standing.

model routing table A table inside each waveset. It records a recommended model tier for the lane orchestrator and for each wave. The README calls it a cost-control surface and not a measured savings claim.

gate-and-proof Name on a listing image in this repo. The line on the image is to accept work only when a

recorded check shows it works. The live site describes that job as `/remeasure`.

handoff-check Name on a listing image in this repo. The line on the image is that a fresh thread checks each handoff and follows a recovery playbook when a check fails. The live site describes that job as the pickup check.

rotation-audit Name on a listing image in this repo for the history audit. The image groups these three names as checks a fresh agent runs from files on disk.

The README also names a self-check loop for proposed skill edits, a set of orchestration fail ids and mechanical prose checks. The checker files and the two docs it points to are not in this repository, so this reading does not restate those id lists.

Three ways the class already described the work

Class description	Wavves term	Where it shows up
A short request into a context store	`/wavves` and the standing home	The class phrase is context store. The product files are the home and `waveset.md`. The request itself stays short.
One bounded stretch of work with a clear finish line	lane, wave and gate	Charter one lane. Number waves from W1 up. Finish checks are the gates and the acceptance checks in the charter.
Freeze traces and check them, instead of summarizing the chat	rotation file, `gate-captures`, pickup check, rotation history audit	Older rotation files never change. A later term checks files, commits and paths. A hand-written summary is not a capture.

Table 2. The three methods as the class has already said them, mapped to terms the wavves sources already use. The class phrases are not renamed product terms.

Class ideas and the feature that does the job

Class idea	Wavves term or function	What the sources say
Chatbots, prompts and prompt methods. A short request in ordinary language.	`/wavves` and a playbook	The entry takes the request, checks the home and runs one playbook. The published examples are short tasks.
Conversation memory and a conversation buffer that keeps recent turns.	No buffer control in these sources	The sources do not describe a buffer of recent turns or a setting that shrinks what the next request sends. Older work sits in files. `/mod-rotate` starts a new term.
Context window, token count and cost. A smaller model and a flagship model.	Listing FAQ and the model routing table	The next term carries nothing forward in the context window. Each waveset records a tier per role. The README publishes no token counts and no savings figure. It does not name a flagship model.
Context window bloat. Long work fills the window and raises token cost and wait time.	lane, wave, standing home, `/mod-rotate`	Long work is chartered as waves with state on disk. A heavy thread is handed to the next term. These sources do not publish a token cost or a wait time.
Context rot. Recaps lose detail, constraints and error notes.	`/remeasure`, pickup check, rotation history audit	A later term checks JSON, logs, claimed commits and claimed paths. A hand-written summary is not a gate capture.
Agents do not need a bigger window or endless recaps for large work.	hydrate from the home	The listing says the next term hydrates from files with nothing carried over in the context window. The README says fresh instances never hydrate from transcripts.
Lasting state lives in files another process can read.	`wavves/` home	`INDEX.md`, the lane waveset, `gate-captures`, rotation files and, in the README, `registry.yml`.
One bounded stretch with a clear finish line.	lane, wave and gate	A lane is one folder. A wave is one round from W1 up. Gates record pass or block. Failed gates are reported as failed.
Write results out as artifacts and later check those files.	`gate-captures` and `/remeasure`	JSON and a log land on disk. Someone other than the author grades the capture. The command that made the capture is named so another person can rerun it.
Separate workers are optional. Artifacts and written directives are required so another process can pick up the work.	waveset, dispatch file, rotation file, runners	The charter, the dispatch and the rotation file are the written record a new term reads. The sources also describe runners for the wave. They do not call those runners optional.
Model comparison and evaluation on quality, cost and latency. A later check against recorded evidence, not a self-score.	grade on the capture and the model routing table	The listing rejects a self-grade. Cost is the tier table. The README says that table is not a measured savings claim. These sources do not report a latency figure.

Class idea	Wavves term or function	What the sources say
Retrieval, embeddings and a vector database. Streamlit apps that host the student tools.	No matching feature in these sources	These sources do not describe embeddings, a vector database or a Streamlit app. The store a later step reads is the `wavves/` folder. Streamlit is how this class hosts its own tools.

Table 3. Each row is a class idea already in use, then the wavves term that does that job in the public pages or the README. Rows with no matching feature say so.

Model tiers recorded before a wave

Role	Model tier	Reason in the README
lane orchestrator	high-reasoning	cross-file planning and gate design
discovery runners	fast	broad search, summaries, inventory
build runners	balanced	bounded edits with local validation
adversarial gate	high-reasoning	defect finding and risk judgment
acceptance gate	high-reasoning	final verification and caveat review

Table 4. The model routing table the README says each `waveset.md` includes. The README says to record runner count, expected file reads, expected gate cost and model tier before execution. A later pass can compare those tiers with token or billing records only when the environment exposes them. This table is not a measurement. Discovery runners are the role whose stated reason includes summaries. Gate evidence is a separate record. A hand-written summary is not a gate capture.

Limits the sources name

Scripts in the package need Python 3 and no other packages. Pickup commit checks and the rotation history audit also need a local git binary. Secret handling through `/set-key` is built for macOS Terminal.app on the machine that holds the secret. This reading does not include a secret, a key name or a command that would set one. `/layover` writes only its report. The README says a cloud agent stays single-repo and that `/layover` does not start one.

The listing says sixty validation cases stay out of the sold package so a buyer runs the script on real captures. Missing or changed hashed artifacts fail under that script. That sixty is the size of a held-out set in the listing, not a score from a class run.

Listing images in this repo name three kits, gate-and-proof, handoff-check and rotation-audit. One image calls them one zip with three kit folders and says a fresh agent checks what the last one claimed, from files on disk. The live site text describes those three jobs inside the one wavves package.

What this reading is drawn from

Included here is the public site text, the public listing text, the repository README and the plain claims on three listing images in this repo. Example counts, sample paths and sample commit ids on those images are not used. The README points at `examples/usage.md`, `docs/public-copy-gates.md`, `docs/purpose-gates.md`, `skills/` and `evals/`. Those paths are not in this repository, so their contents are not restated. No token total, dollar savings figure or latency figure is added, because the sources do not give one.

This is a class reading for review. It is not a lab submission and it is not the sold skill package.